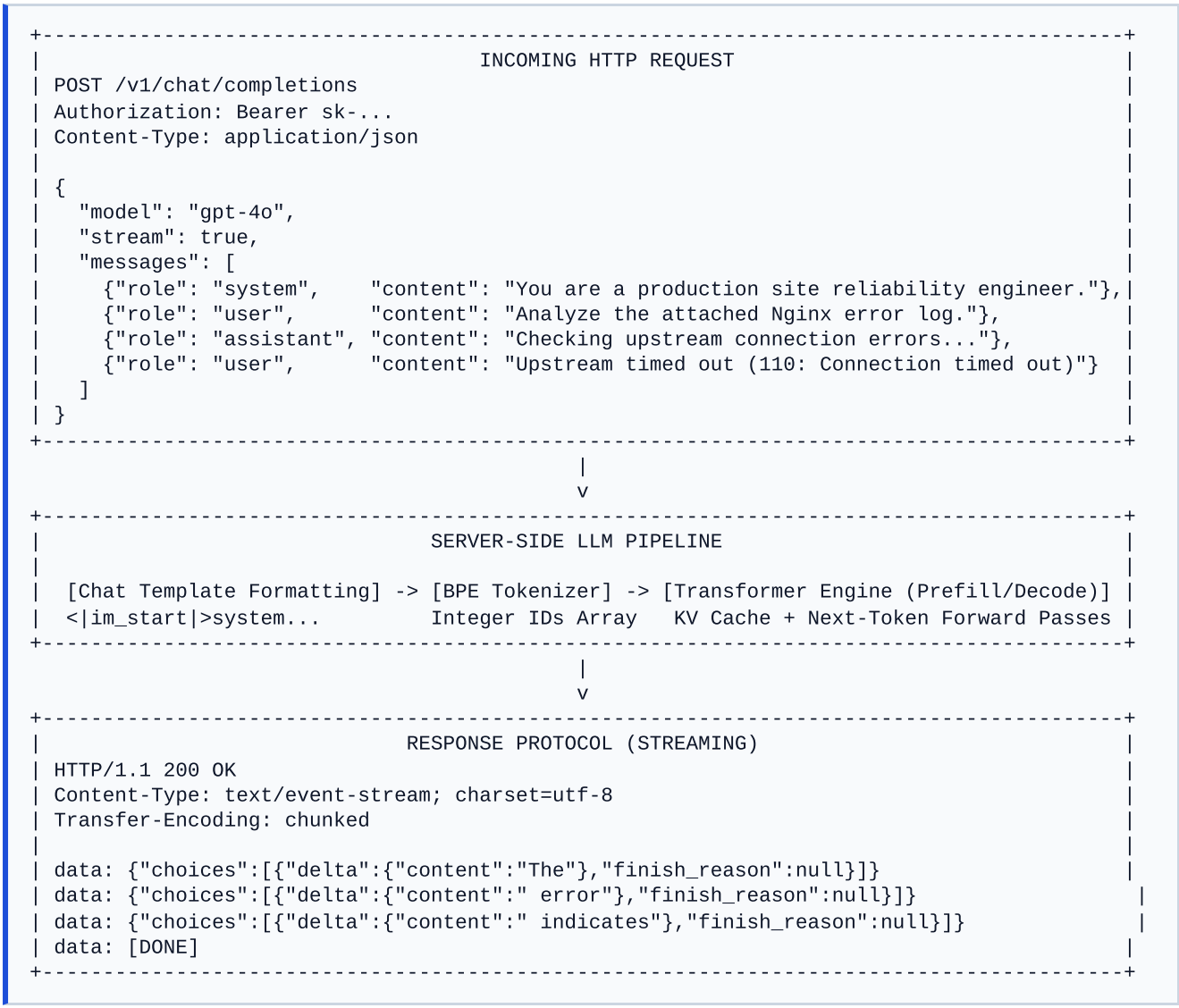
TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 6: Production LLM APIs & State Management

Module 6 covers the systems, networking, and economic layers of deploying Large Language Models into production environments: API contracts, streaming protocols, reliability engineering, traffic budgeting, and stateless context orchestration.

6.1 Live LLM APIs: Endpoint Architecture, Message Roles, and SSE

Commercial and open-source inference servers (OpenAI, Anthropic, vLLM, Ollama, Hugging Face TGI) expose HTTP REST endpoints modeled around the **Chat Completions** standard.



1. The Chat Completions API Specification

The industry standard endpoint is `POST /v1/chat/completions`. Under the hood, the raw foundation model only understands a flat 1D sequence of tokens. The inference engine applies a model-specific **Chat Template** (such as ChatML or LLaMA-3 Jinja templates) using special delimiting tokens (e.g., `<|im_start|>role ... <|im_end|>`) to convert the structured JSON message array into a single text prompt before tokenization.

2. Message Roles and Their Architectural Functions

Modern APIs support five core roles:

Role	Semantic Meaning & Engine Treatment	Production Best Practice
`system`	Sets global behavioral constraints, tone, output format rules, and guardrails. Processed at the very beginning of the prompt.	Keep static across requests to maximize server-side **prefix/prompt caching**.
`developer`	Introduced in newer frontier architectures (such as OpenAI o-series) to prioritize operator constraints over user instructions, mitigating prompt injection.	Use for structural invariants, schema definitions, and system-level rules.
`user`	The raw input provided by the client, application end-user, or external upstream caller. Can carry multimodal payloads (text + base64/URL images and audio).	Sanitize untrusted input; isolate from instructions using clear XML/Markdown delimiters (`<input>...</input>`).
`assistant`	Prior model completions, or synthetic responses inserted by the developer to provide **few-shot conversational exemplars** or pre-fill completion prefixes.	Pre-fill valid JSON brackets (e.g., `{"status":`) to steer the model into strict JSON format without syntax errors.
`tool`	Returns the raw stringified output of an external client-executed function call back to the model, matching a prior `tool_call_id`.	Always return deterministic, minified payloads (strip unnecessary JSON keys) to minimize context window consumption.

3. Key Request Parameters

```
payload = {
    "model": "gpt-4o",  # Underlying checkpoint or routing alias
    "messages": [
        {"role": "system", "content": "You are a concise data extractor."},
        {"role": "user", "content": "Extract order ID from: Invoice #98421"}
    ],
    "temperature": 0.1,                # Near-zero for deterministic extraction
    "top_p": 0.95,                     # Nucleus sampling boundary
    "max_completion_tokens": 150,      # Upper bound on output tokens generated
    "stop": ["\n\n", "###"],          # Explicit token strings that halt generation
    "stream": True,                    # Emits chunked Server-Sent Events
    "response_format": {               # Enforces valid JSON via constrained decoding
        "type": "json_object"
    }
}
```

6.2 Streaming Token Generation: Network Mechanics, SSE vs. WebSockets

In non-streaming requests (`stream=False`), the client sends an HTTP POST request and the connection sits idle while the LLM executes the prefill phase followed by hundreds of sequential decode forward passes. The entire payload is returned only after generation finishes, driving Time-To-First-Token (TTFT) up into multi-second territory.

Streaming bypasses this latency bottleneck by pipelining each newly generated token over an open transport stream the moment it is sampled.

```
[Client]                                                    [LLM Gateway / Engine]
   |                                                                  |
   |--- 1. POST /v1/chat/completions (stream=true) ----------------------->|
   |                                                                  | (Prefill
Prompt)
   |<-- 2. HTTP/1.1 200 OK (Content-Type: text/event-stream) -------------| (Decode Token
1)
   |                                                                  |
   |<-- 3. data: {"choices":[{"delta":{"content":"Hello"}}]} -------------| (TTFT achieved)
   |                                                                  | (Decode Token
2)
   |<-- 4. data: {"choices":[{"delta":{"content":" world"}}]} ------------|
   |                                                                  | (Decode Token
3)
   |<-- 5. data: {"choices":[{"delta":{"content":"!"}}]} -----------------|
   |                                                                  |
   |<-- 6. data: [DONE] --------------------------------------------------| (Stream Closed)
   |                                                                  |
```

1. Server-Sent Events (SSE) Wire Format

SSE is a unidirectional, text-based protocol layered directly on standard HTTP (defined under the W3C EventSource specification).

- The server responds with headers:

  ```
  HTTP/1.1 200 OK
  Content-Type: text/event-stream; charset=utf-8
  Cache-Control: no-cache
  Connection: keep-alive
  Transfer-Encoding: chunked
  ```

- Every chunk consists of a `data:` prefix, followed by a JSON payload containing the delta token, terminated by **two newlines (`\n\n`)**:

  ```
  data: {"id":"chatcmpl-101","choices":[{"index":0,"delta":
  {"content":"System"},"finish_reason":null}]}

  data: {"id":"chatcmpl-101","choices":[{"index":0,"delta":{"content":"
  ready"},"finish_reason":null}]}

  data: [DONE]
  ```

- **Termination Marker:** The literal string `data: [DONE]\n\n` notifies the client that the generation loop has reached `<eos>` or a stop condition, allowing the client parser to cleanly close the socket.

2. SSE vs. WebSockets for LLM Streaming

Dimension	Server-Sent Events (SSE)	WebSockets
Directionality	**Unidirectional (Server → Client).** Requests initiate via standard HTTP POST; responses stream back down.	**Bidirectional (Full Duplex).** Both client and server can send frames concurrently over a single socket.
Transport Protocol	Standard HTTP/1.1 or HTTP/2 streams.	Distinct `ws://` or `wss://` protocol initiated via HTTP upgrade handshake.
Proxy & Firewall Traversal	Seamless. Traversals through corporate proxies, CDNs, Cloudflare, and AWS ALBs treat it as normal HTTP.	Requires specialized connection-handling, sticky sessions, and explicit WebSocket upgrade proxy rules.
Reconnection Semantics	Native browser auto-reconnect via `Last-Event-ID`.	Custom application-level heartbeat/ping-pong and reconnection logic required.
Production Use Case	Standard chat interfaces, agent step streaming, code generation consoles.	Real-time duplex voice engines (e.g., GPT-4o Realtime API), live screen/video streaming agents.

3. Client-Side Consumption & Buffer Assembly (Python)

Because TCP packets can fragment arbitrary byte boundaries, a multi-byte UTF-8 character (e.g., an emoji taking 4 bytes) may occasionally be split across consecutive SSE network packets. A production client parser must buffer and decode text cleanly:

```
import httpx
import json

async def stream_chat_completion(api_key: str, prompt: str):
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": "gpt-4o",
        "messages": [{"role": "user", "content": prompt}],
        "stream": True,
        "stream_options": {"include_usage": True}  # Emits usage token counts in final chunk
    }

    async with httpx.AsyncClient(timeout=httpx.Timeout(60.0, connect=10.0)) as client:
        async with client.stream(
            "POST",
            "https://api.openai.com/v1/chat/completions",
            headers=headers,
            json=payload
        ) as response:
            if response.status_code != 200:
                error_body = await response.aread()
                raise RuntimeError(f"API Error {response.status_code}:
{error_body.decode()}")

            # Iterate over SSE lines split cleanly on line breaks
            async for line in response.aiter_lines():
                line = line.strip()
                if not line or not line.startswith("data: "):
                    continue

                raw_data = line[len("data: "):]
                if raw_data == "[DONE]":
                    break

                chunk = json.loads(raw_data)
                choices = chunk.get("choices", [])
                if choices and "delta" in choices[0]:
                    token = choices[0]["delta"].get("content", "")
                    if token:
                        print(token, end="", flush=True)

                # Extract final usage chunk if available
                if "usage" in chunk and chunk["usage"] is not None:
                    usage = chunk["usage"]
                    print(f"\n[Usage: {usage['prompt_tokens']} prompt,
{usage['completion_tokens']} completion]")
```

6.3 Authentication, Rate Limits, Retry/Backoff, and Resiliency

Deploying LLMs at scale demands defensive infrastructure around network boundaries, quota throttles, and downstream provider outages.

1. Authentication & Enterprise Secret Hygiene

- **Transport Security:** All communication occurs over TLS 1.3. The API key is injected as a bearer token in the HTTP Authorization header:

```
Authorization: Bearer sk-proj-xxxxxxxxxxxxxxxxxxxxxxxx
OpenAI-Organization: org-enterprise-core
OpenAI-Project: proj-data-pipelines
```

- **Runtime Secret Injection:** Never hardcode credentials in source control or Docker layers. Secrets should be mounted into local memory from `.env` files during local development, or retrieved dynamically from secure secret managers (AWS Secrets Manager, HashiCorp Vault) into runtime environment variables (`os.environ["OPENAI_API_KEY"]`).

2. Rate Limiting Mechanics (RPM vs. TPM)

Inference providers enforce rate limits using two independent thresholds:

1. **Requests Per Minute (RPM):** Mitigates connection exhaustion and load balancer saturation on HTTP reverse proxies.
2. **Tokens Per Minute (TPM):** Mitigates GPU memory and compute saturation. TPM accounts for both **estimated prompt tokens + max output tokens requested** upon request arrival, even before decoding begins.

```
Rate Limiter Logic (Token Bucket / Sliding Window):
Incoming Request (Prompt: 4,000 tokens, max_completion: 1,000 tokens)
  │
  ├─ Check 1: Current RPM + 1 <= RPM_Limit?
  │      └─ NO  --> HTTP 429 (Rate limit reached for requests)
  │      └─ YES --> Proceed to Check 2
  │
  └─ Check 2: Current TPM + 5,000 <= TPM_Limit?
         └─ NO  --> HTTP 429 (Rate limit reached for tokens)
         └─ YES --> Deduct tokens from bucket, dispatch forward pass
```

Providers signal rate limit status in the HTTP response headers:

```
x-ratelimit-limit-requests: 5000
x-ratelimit-remaining-requests: 4998
x-ratelimit-reset-requests: 24ms
x-ratelimit-limit-tokens: 450000
x-ratelimit-remaining-tokens: 432100
x-ratelimit-reset-tokens: 2.38s
retry-after: 3
```

3. HTTP Error Taxonomy & Triage

Status Code	Error Category	Root Cause	Handling Strategy
`400 Bad Request`	Client Error	Malformed JSON schema, context length overflow (`prompt tokens > context window`).	**Do NOT retry.** Fail fast, truncate prompt, or correct request schema.
`401 Unauthorized`	Auth Error	Missing, invalid, or revoked API key.	**Do NOT retry.** Alert ops / rotate credentials immediately.
`429 Too Many Requests`	Throttle / Quota	Exceeded RPM/TPM limit, or depleted monthly prepaid account credits.	Inspect response body: if quota exceeded, halt; if rate limit, parse `retry-after` header and back off.
`500 / 502 / 503`	Server Error	Gateway failure, internal timeout, or infrastructure crash.	**Retry.** Eligible for transient backoff logic.
`529 Overloaded`	Capacity Error	Provider GPU cluster has insufficient capacity to serve request.	**Retry with failover.** Route to alternative regional endpoint or fallback model.

4. Exponential Backoff with Full Jitter

When an API returns a transient error (`429` , `500` , `503` , `529`), thousands of concurrent client workers must avoid retrying simultaneously. Identical delay curves trigger a **thundering herd problem** that keeps the upstream provider permanently saturated.

The mathematical formulation for **Exponential Backoff with Full Jitter** is:

> $$t_{\text{temp}} = \min(t_{\max}, t_{\text{base}} \times 2^{\text{attempt}}), \quad t_{\text{sleep}} = \text{Uniform}(0, t_{\text{temp}})$$

```
import asyncio
import random
import httpx

async def call_llm_with_resilience(client: httpx.AsyncClient, url: str, headers: dict,
payload: dict, max_retries: int = 5):
    base_delay = 1.0   # Initial sleep base (1 second)
    max_delay = 32.0   # Cap backoff ceiling at 32 seconds

    for attempt in range(max_retries):
        try:
            response = await client.post(url, headers=headers, json=payload, timeout=30.0)

            # Success
            if response.status_code == 200:
                return response.json()

            # Non-retryable client errors
            if response.status_code in [400, 401, 403]:
                raise RuntimeError(f"Fatal Client Error {response.status_code}:
{response.text}")

            # Retryable status codes (429, 500, 502, 503, 529)
            if response.status_code in [429, 500, 502, 503, 529]:
                # Check if provider explicitly suggested a retry delay
                retry_after_header = response.headers.get("Retry-After")
                if retry_after_header:
                    sleep_duration = float(retry_after_header)
                else:
                    # Exponential Backoff with Full Jitter
                    calculated_delay = min(max_delay, base_delay * (2 ** attempt))
                    sleep_duration = random.uniform(0, calculated_delay)

                print(f"[Warning] HTTP {response.status_code}. Retrying in
{sleep_duration:.2f}s (Attempt {attempt + 1}/{max_retries})...")
                await asyncio.sleep(sleep_duration)
                continue

        except (httpx.ConnectTimeout, httpx.ReadTimeout, httpx.ConnectError) as exc:
            calculated_delay = min(max_delay, base_delay * (2 ** attempt))
            sleep_duration = random.uniform(0, calculated_delay)
            print(f"[Network Error] {exc}. Retrying in {sleep_duration:.2f}s...")
            await asyncio.sleep(sleep_duration)

    raise TimeoutError(f"Exhausted all {max_retries} retry attempts without successful
response.")
```

5. Circuit Breaker Pattern & Multi-Provider Fallbacks

In mission-critical production pipelines, relying on a single upstream provider is an operational single-point-of-failure. Enterprise gateways (e.g., LiteLLM Proxy, Portkey, Kong AI Gateway) implement the **Circuit Breaker Pattern**:

- **Closed State:** Requests route directly to Primary Model (e.g., OpenAI GPT-4o).
- **Open State:** If the rolling error rate exceeds 25% over a 60-second window, the circuit breaker trips to "Open" and immediately diverts traffic to a Secondary Provider (e.g., Anthropic Claude 3.5 Sonnet on AWS Bedrock or self-hosted vLLM LLaMA 3.1) without waiting for HTTP timeouts.
- **Half-Open State:** After a cooldown window (e.g., 30s), a small sample of canary requests (5%) are routed back to the Primary to check for recovery before closing the circuit completely.

6.4 Token-Based Pricing: Cost Formulation & Production Budget Modeling

Unlike traditional microservices billed on compute instance runtime (CPU/RAM hours), LLM production costs scale purely on token volume and context reuse.

1. The Real-World Billing Equation

LLM APIs feature asymmetric billing: **output generation tokens cost *3*× to *5*× more than input prompt tokens** because sequential decoding ties up memory-bandwidth-bound GPU cores. Furthermore, modern providers offer **prompt caching discounts** for matching prompt prefixes stored in the KV cache.

$$\text{Total Cost} = \frac{(N_{\text{uncached_in}} \cdot P_{\text{in}}) + (N_{\text{cached_in}} \cdot P_{\text{cached}}) + (N_{\text{out}} \cdot P_{\text{out}}) + (N_{\text{reasoning}} \cdot P_{\text{out}})}{1{,}000{,}000}$$

Where:

- $N_{uncached_in}$: Fresh input tokens that required prefill matrix multiplication.
- N_{cached_in}: Input tokens matched against an active KV cache (typically discounted by 50% to 90%).
- N_{out}: Visible tokens generated in the completion.
- $N_{reasoning}$: Internal hidden thinking tokens generated by reasoning models (billed at the output token rate).
- P_{in}, P_{cached}, P_{out}: Price per 1 Million tokens set by the provider.

2. End-to-End Production Capacity & Spend Calculation

Let's model an enterprise AI Assistant serving a customer base with the following workload:

- **User Base:** 10,000 Daily Active Users (DAU).
- **Usage:** Each user performs an average of 5 query turns per day (50,000 requests/day).
- **Prompt Profile (RAG setup):**
  - Static System Prompt & Tool Schemas: 1,000 tokens (90% cache hit rate across users).
  - Dynamic RAG Context & User Input: 2,000 tokens (uncached).
  - Average Assistant Completion: 400 output tokens.
- **Selected Model:** Flagship Tier (e.g., Input: $2.50 / 1M, Cached: $1.25 / 1M, Output: $10.00 / 1M).

Step-by-Step Economic Breakdown (Per Day):

1. **Input Tokens per Day:**
   - Total Requests = *50,000*
   - Cached Input Tokens = *50,000* × *1,000* tokens = *50,000,000* tokens
   - Uncached Input Tokens = *50,000* × *2,000* tokens = *100,000,000* tokens
2. **Output Tokens per Day:**
   - Total Output Tokens = *50,000* × *400* tokens = *20,000,000* tokens
3. **Daily Cost Calculation:**

> Cached Input Cost = (50,000,000 × $1.25) / 1,000,000 = $62.50

> Uncached Input Cost = (100,000,000 × $2.50) / 1,000,000 = $250.00

> Output Token Cost = (20,000,000 × $10.00) / 1,000,000 = $200.00

> **Total Daily Spend** = $62.50 + $250.00 + $200.00 = **$512.50 / day** (**Monthly Run Rate (30 Days): $15,375.00 / month**)

3. FinOps Optimization Strategies

1. **Prompt Structure Invariance (Cache Alignment):** Providers hash prompt prefixes from left to right. If you place a dynamic timestamp or random user ID at the very top of your system prompt, you invalidate the prefix hash and destroy 100% of prompt caching savings. Keep static content at the top and variable dynamic text at the bottom.
2. **Model Cascading / Tiered Routing:** Route 80% of routine categorization and extraction queries to a lightweight Small Language Model (SLM, costing $0.15 / 1M), escalating only the 20% most complex queries to the $10.00 / 1M model. This cuts overall blended API spend by 60–75%.

3. **OpenAI / Anthropic Batch API:** For asynchronous workloads (offline evaluation, overnight bulk document extraction), use the Batch API endpoint. Jobs complete within 24 hours at an automatic **50% flat discount**.

6.5 Multi-Turn Conversation State & Stateless Architecture

A fundamental property of LLM infrastructure is that **the API endpoint is 100% stateless**. The server maintains zero persistent session memory between distinct HTTP calls.

```
Turn 1:
Client sends: [Sys, User_1]
Server returns: Assistant_1
Server drops all activations from RAM.

Turn 2:
Client sends: [Sys, User_1, Assistant_1, User_2]
Server returns: Assistant_2
Server drops all activations from RAM.

Turn 3:
Client sends: [Sys, User_1, Assistant_1, User_2, Assistant_2, User_3]
Server returns: Assistant_3
```

To create the illusion of continuous memory, the **client application must manage session state and re-transmit the entire conversation history on every single turn**.

1. The $O(K^2)$ Context Explosion Problem

If each turn in a conversation contains an average of L tokens, the cumulative volume of prompt tokens re-transmitted and billed over K turns is:

$$\text{Total Billed Tokens} = \sum_{k=1}^{K} k \cdot L = L \cdot \frac{K(K+1)}{2} = O(K^2)$$

By Turn 20, a conversation where each turn is 1,000 tokens has billed over **210,000 prompt tokens**, consuming network bandwidth, driving up latency, and threatening context window limits.

2. State Management Strategies in Production

Production architectures deploy four distinct context-management strategies:

```
[Full Chat History] ──► [Pruning / Windowing Engine] ──► [Compiled Active Context]
                                  │                              │
                                  ├── Strategy A: Rolling FIFO   ├── System Prompt
                                  ├── Strategy B: Semantic Summary ├── Summarized Past
                                  └── Strategy C: Vector RAG Recall └── Last N Clean
Turns
```

Strategy A: Rolling FIFO Window (Sliding Window)

- **Mechanics:** Always preserve the static System Prompt (`messages[0]`). Retain only the last *N* dialogue turns (e.g., last 6 messages). Evict older messages from the array.
- **Advantage:** Low compute overhead; keeps token consumption bounded.
- **Failure Mode:** Complete amnesia regarding instructions or constraints stated by the user earlier in the session.

Strategy B: Recursive / Hierarchical Summarization

- **Mechanics:** When the message history exceeds a threshold (e.g., 10 turns), a background SLM task summarizes the oldest 6 turns into a structured paragraph (*"The user is debugging a Docker network timeout on Ubuntu 22.04..."*). The oldest turns are pruned, and the summary is injected into a dedicated context block:

```
[
  {"role": "system", "content": "You are a support agent.\n\nSummary of prior
discussion:\n<summary>User initialized a PostgreSQL 15 database on port 5432 and verified
container health.</summary>"},
  {"role": "user", "content": "Now how do I connect my FastAPI backend to it?"}
]
```

- **Advantage:** Retains key context across long multi-hour sessions without token bloat.

Strategy C: Semantic Episodic Memory (Memory RAG)

- **Mechanics:** The application extracts key facts, preferences, and entities from each turn, storing them as discrete records in an external database (PostgreSQL with `pgvector`, Redis, or SQLite).
- On new turns, the system queries the memory store for facts semantically relevant to the current user prompt, dynamically injecting only relevant memories into the active context.

3. Production Multi-Turn State Orchestration (Python + Redis)

```
import json
import redis
from typing import List, Dict

class ConversationStateManager:
    """
    Manages stateless multi-turn chat sessions backed by Redis.
    Implements System-pinned Rolling FIFO context windowing.
    """
    def __init__(self, redis_client: redis.Redis, session_id: str, max_turns: int = 6):
        self.r = redis_client
        self.session_key = f"chat:session:{session_id}"
        self.max_turns = max_turns  # Max conversational turns retained (User + Assistant
pairs)
        self.system_prompt = {
            "role": "system",
            "content": "You are an enterprise support copilot. Provide concise technical
answers."
        }

    def append_message(self, role: str, content: str):
        message = json.dumps({"role": role, "content": content})
        self.r.rpush(self.session_key, message)
        # Set 24-hour Time-To-Live (TTL) on session key to prevent storage leaks
        self.r.expire(self.session_key, 86400)

    def get_compiled_context(self) -> List[Dict[str, str]]:
        # Fetch raw serialized messages from Redis
        raw_messages = self.r.lrange(self.session_key, 0, -1)
        history = [json.loads(m.decode("utf-8")) for m in raw_messages]

        # Apply Sliding Window: Retain only the most recent (max_turns * 2) messages
        window_size = self.max_turns * 2
        trimmed_history = history[-window_size:] if len(history) > window_size else history

        # Prepend the immutable system prompt
        return [self.system_prompt] + trimmed_history

    def clear_session(self):
        self.r.delete(self.session_key)
```

Production Architecture Synthesis

Production Component	Architectural Choice	Key Engineering Metric / SLA
API Transport	HTTP/2 SSE (`text/event-stream`)	TTFT < 400ms
Resilience & Retries	Exponential Backoff with Full Jitter + Circuit Breakers	Zero unhandled 5xx cascades; ≤ 3 retries
Throughput & Limits	Token Bucket algorithm with local proactive TPM tracking	Stay below 85% of assigned upstream TPM limits
Context Hygiene	System-pinned rolling window + sliding prefix caching	Maximize KV-cache prefix hits (> 70% cached tokens)
Economic FinOps	Model cascading (SLM triage → Flagship reasoning) + Batch API	Minimizing cost per resolved user journey